AIMP: AI Mesh Protocol

Design and Evaluation of a Serverless
Merkle-CRDT Protocol for Edge Agent Synchronization

Fabrizio Salmi

v0.1.0 — March 2026

https://github.com/fabriziosalmi/aimp

Abstract

We present AIMP (AI Mesh Protocol), a serverless networking protocol for resilient state synchronization between autonomous agents in fragmented, low-bandwidth networks. AIMP combines a Merkle-DAG CRDT with Noise Protocol XX encrypted gossip, Ed25519 zero-trust identity, and BFT quorum consensus. We use bounded model checking in TLA+ to verify three safety properties with up to 3 nodes (101 million states explored, zero violations), uncovering and fixing two correctness bugs in the process. Performance evaluation shows 96K mutations/sec on a 5-node cluster, sub-millisecond convergence, and resilience up to 60% packet loss. With the optional `ring` cryptographic backend, AIMP achieves 129K mutations/sec — 1.37 × faster than Automerge v0.7 on mutation throughput and 2.4× faster on 2-replica merge, while providing Ed25519 cryptographic integrity that Automerge lacks. Compared to Yrs (Yjs), AIMP achieves competitive merge latency (0.48 ms vs 0.38 ms) despite cryptographic overhead. A batch signing mode amortizes Ed25519 cost via Merkle trees, reaching 891K mutations/sec (1.41× faster than Yrs) with full cryptographic integrity. A gossip fan-out delta-sync prototype reduces 100-node convergence from 5.4 seconds to 617 ms. All benchmarks are fully reproducible via a single script. The reference implementation compiles to a single static binary under 10 MB targeting ARM64, ARMv7, and x86_64.

1 Introduction

State synchronization in distributed systems traditionally relies on leader-based consensus protocols such as Raft [1] or PBFT [2], which require stable quorums and assume reliable network connectivity. These assumptions break down at the network edge: industrial sensor networks, autonomous vehicle fleets, disaster response teams, and IoT deployments routinely experience network partitions, high latency, and intermittent connectivity.

AIMP addresses this gap by combining several well-established techniques from distributed systems research into a single, edge-optimized protocol:

1. **Merkle-DAG CRDT** for conflict-free state synchronization with cryptographic integrity, following Kleppmann and Howard [3].
2. **Gossip-based dissemination** with O(1) deduplication and TTL-bounded propagation [4].
3. **Noise Protocol XX** for authenticated, forward-secret encrypted peer sessions [5].
4. **BFT quorum voting** for deterministic decision verification across untrusted peers.

5. **Pluggable Decision Engine** with hot-reloadable rules for deterministic edge logic.

Contributions. This paper makes the following contributions:

- A protocol design combining Merkle-CRDTs, gossip, and BFT quorum for edge agent synchronization (3,000 lines of Rust, 18 integration tests, 2 property-based tests).
- TLA+ formal verification of three safety properties, with two real bugs discovered and fixed in both the specification and the Rust implementation.
- Comprehensive performance evaluation: micro-benchmarks, 5-node system benchmarks, network impairment simulation, cross-platform ARM64 profiling, and quantitative comparison with Automerge v0.7.
- Equivocation slashing adapted to Merkle-DAG CRDTs: Byzantine detection with decoupled data/control planes (CRDT merge preserved, consensus layer isolates attacker).
- A fully reproducible benchmark suite (`./benchmarks/run_all.sh`).

2 Related Work

CRDTs. Conflict-free Replicated Data Types [6] provide mathematically guaranteed convergence without coordination. AIMP uses a state-based Merkle-DAG CRDT where the merge operation is set union — commutative, associative, and idempotent by construction.

Merkle-CRDTs. Building on Merkle's hash trees [7], Kleppmann and Howard [3] formalized Merkle trees combined with CRDTs. AIMP extends this with epoch-based garbage collection to bound memory growth.

Byzantine Consensus. Castro and Liskov's PBFT [2] established practical BFT. AIMP uses a simplified quorum voting scheme for decision verification, reducing message complexity.

Gossip Protocols. Epidemic dissemination [4] provides probabilistic reliability. AIMP's gossip layer uses bounded deduplication (FxHashSet + VecDeque ring buffer) and TTL-based replay protection.

Noise Protocol Framework. The Noise Protocol Framework [5] provides authenticated key exchange. AIMP uses the XX handshake pattern with BLAKE3 for mutual authentication and forward secrecy.

Vector Clocks. Mattern's vector clocks [8] capture causal ordering. AIMP attaches vector clocks to DAG nodes, complementing the topological ordering of the Merkle-DAG.

3 System Design

3.1 Architecture Overview

Layer	Component
Transport	UDP socket with per-peer token bucket rate limiting
Security	Noise Protocol XX (ChaCha20-Poly1305 + BLAKE3)
Authentication	Ed25519 signature verification firewall
Serialization	MessagePack envelope (version, opcode, TTL, vector clock)
Dissemination	Gossip with O(1) dedup (FxHashSet + VecDeque)
State	Merkle-DAG CRDT with slab/arena (SmallVec parents)
Persistence	redb with ChaCha20Poly1305 encryption at rest
Consensus	BFT quorum voting for decision verification
Application	Pluggable Decision Engine with hot-reload
Observability	Prometheus metrics + structured event logging

Table 1: AIMP protocol stack.

3.2 Merkle-DAG CRDT

The core state structure is a directed acyclic graph where each node contains: parent hash references (`SmallVec<[Hash32; 2]>`, stack-allocated for the common case of ≤ 2 parents), a BLAKE3 data hash, an Ed25519 signature, a vector clock (`BTreeMap<String, u64>`, sorted for deterministic hashing), and optional decision evidence.

The merge operation is set union over DAG nodes. Given replicas A and B, the merged state $A \cup B$ contains all nodes from both, with the frontier (heads) recomputed as nodes with no children. This is commutative, associative, and idempotent.

The Merkle root is computed by sorting frontier hashes and streaming them through BLAKE3 (zero allocation). The root is cached and only recomputed on frontier change (invalidation-on-write), making reads O(1).

3.3 Actor Model

The CRDT engine runs as a Tokio actor via `mpsc` channels, eliminating lock contention. After merge, heads are recomputed from the full arena to handle out-of-order message delivery correctly — a fix discovered through TLA+ verification (Section 5.2).

3.4 Epoch-Based Garbage Collection

After a configurable mutation threshold, mark-and-sweep GC performs BFS from heads up to `DAG_HISTORY_DEPTH`, then removes unreachable nodes from the slab arena.

4 Decision Engine

AIMP is designed as the synchronization and consensus layer for distributed AI agents. While the protocol itself is domain-agnostic, it includes a pluggable `DecisionEngine` trait explicitly designed to wrap deterministic edge AI models (e.g., via WebAssembly sandboxing for cross-architecture reproducibility). The `engine_hash()` method provides a deterministic version identifier enabling the BFT quorum to verify that all nodes used identical decision logic — a requirement for consensus on AI inference output.

The reference implementation provides a `RuleEngine` that evaluates keyword-matching rules from a hot-reloadable JSON file, demonstrating the consensus mechanism. ML backend integration (e.g., TensorFlow Lite or ONNX Runtime compiled to Wasm) is deferred to future work.

5 Formal Verification

We specify three safety properties in TLA+ (`formal/AimpCrdtConvergence.tla`) and verify them using TLC bounded model checking. While TLC provides exhaustive state exploration within the configured bounds rather than a generalized proof for arbitrary N, exhausting 101 million states provides high empirical confidence in protocol safety.

Property	**Statement**	**Status**
Convergence	Same store ⇒ same Merkle heads	Verified
QuorumSafety	Quorum reached ⇒ unique decision	Verified
QuorumLiveness	All nodes vote same ⇒ threshold met	Verified

Table 2: TLA+ properties verified with TLC model checker.

We verify with two model configurations to demonstrate scalability:

5.1 TLC Model Checking Results

Metric	**2 nodes, 3 mut**	**3 nodes, 2 mut**
States generated	46,063	101,282,509
Distinct states	9,558	12,886,344
State graph depth	16	22
Time	< 1 s	77 s
Invariant violations	0	0
FP collision prob.	1.9×10^{-11}	1.3×10^{-5}

Table 3: TLC results for two model configurations (10 parallel workers, Apple Silicon). Both pass all three invariants with zero violations.

The 3-node configuration explores over 101 million states. A 3-node × 3-mutation configuration exceeds 280 million states in 5 minutes without completing, demonstrating the exponential growth of the state space.

5.2 Bugs Found During Verification

TLA+ verification uncovered two correctness bugs present in both the specification and the Rust implementation:

1. **Out-of-order Receive**: Incremental head update `heads = (heads union {msg.id}) without msg.parents` produced incorrect frontiers when messages arrived out of causal order. Fix: recompute heads from the full store after each merge.
2. **Quorum double-voting**: A node could vote for d_1 then d_2 on the same prompt, violating QuorumSafety. Fix: enforce one vote per prompt per node, not per (prompt, decision).

Both were fixed in the TLA+ spec and Rust code. The corrected model passes all invariants with zero violations.

6 Performance Evaluation

All benchmarks run on Apple Silicon (M-series), single-threaded, `--release` with `target-cpu=native`. Results are fully reproducible via `./benchmarks/run_all.sh`.

6.1 Micro-Benchmarks (Criterion)

Operation	**Median**	**Throughput**
append_mutation (100 ops)	41.8 µs	2.4M mut/s
get_merkle_root (cached)	4.8 ns	O(1)
BLAKE3 hash (1 KB)	925 ns	1.08 GB/s
MessagePack ser / de	204 / 210 ns	—
Ed25519 sign (ring)	9.3 µs	108K ops/s
Ed25519 verify	25.0 µs	40K ops/s

Table 4: Criterion micro-benchmarks (fast-crypto mode, Apple Silicon).

6.2 Hot-Path Profiling

To identify optimization targets, we profile each step of the mutation hot path over 10,000 iterations:

Step	Time	%
Ed25519 sign	8.2 µs	88.5%
append_mutation (DAG + arena)	536 ns	5.8%
rmp_serde serialize	149 ns	1.6%
BTreeMap vclock	121 ns	1.3%
BLAKE3 hash	71 ns	0.8%
format! (benchmark overhead)	38 ns	0.4%
Total	**9,273 ns**	**100%**

Table 5: Hot-path breakdown (ring backend, target-cpu=native). Ed25519 dominates at 88.5%. Efficiency: 88.5% of theoretical maximum.

This confirms that further optimization of the non-crypto path (currently 1.1 µs) yields diminishing returns. The protocol is within 11.5% of the hardware limit imposed by Ed25519.

6.3 System-Level Benchmarks

5-node in-process cluster with anti-entropy sync:

Scenario	Result
Throughput (5 nodes × 1000 mutations)	96,289 mut/s
Convergence (5 divergent nodes, 250 DAG each)	0.68 ms (1 round)
Partition/Merge (2 groups, 30 mut/group)	0.21 ms (1 round)
Crypto cost (sign + verify)	45.0 µs/msg
Crypto budget at rate_limit=50/sec	0.23%

Table 6: System-level benchmarks (fast-crypto mode, Apple Silicon).

Convergence in a single anti-entropy round confirms the CRDT property: one exchange is sufficient for full state reconciliation.

6.4 Network Impairment Simulation

Condition	Converged	Rounds
Baseline (0% loss)	Yes	1
10% packet loss	Yes	1–2
30% packet loss	Yes	2
50% packet loss	Yes	2–3
20% loss + 100ms lat + 30ms jitter	Yes	2
Partition (50R) then merge + 20% loss	Yes	1
60% packet loss (stress)	Yes	4
80% packet loss (stress)	Partial	3–4
90%+ packet loss	No	—

Table 7: Convergence under simulated network impairment (5 nodes, 50 mutations/node).

AIMP converges within 1–2 rounds up to 50% loss, and degrades gracefully through 60–80%. Above 90%, pairwise exchange probability drops below the dissemination threshold — a fundamental property of gossip protocols.

6.5 Cross-Platform: Resource-Constrained ARM64

Docker ARM64 Linux with resource limits matching edge hardware:

Metric	macOS ARM64	1C/1GB (Pi 4)	1C/256MB (Pi Zero)
Throughput (mut/s)	96,289	24,802	29,709
Convergence (ms)	0.68	3.06	1.30
Ed25519 sign (µs)	9.3	16.2	15.1
Crypto total (µs/msg)	45.0	50.8	60.3
Max msg/sec	22,232	19,695	16,573
Budget at 50/s	0.23%	0.25%	0.30%

Table 8: Cross-platform: macOS native vs resource-constrained ARM64 Linux.

Crypto cost increases 2× under constraints, but throughput ceiling (> 16K msg/s) remains orders of magnitude above the rate limit. AIMP is viable on edge hardware.

7 Comparison with Automerge and Yrs

We compare against two widely-used Rust CRDT libraries: Automerge v0.7 [3] (operation-based CRDT) and Yrs v0.25 (the Rust port of Yjs, a high-performance text CRDT). All benchmarked on the same hardware with `target-cpu=native`.

Benchmark	**AIMP (ring)**	**Automerge**	**Yrs (Yjs)**
Mutation (1000 ops)	129K ops/s	94K ops/s	632K ops/s
2-replica merge	0.48 ms	1.17 ms	0.38 ms
5-replica merge	2.16 ms	3.89 ms	—

Table 9: AIMP vs Automerge v0.7 vs Yrs v0.25 (Apple Silicon, target-cpu=native). AIMP includes Ed25519 signing; Automerge and Yrs do not.

Yrs achieves the highest mutation throughput (632K ops/s) as a batch-optimized text CRDT with no cryptographic overhead. AIMP with `ring` (129K ops/s) outperforms Automerge (94K ops/s) by 1.37× while including Ed25519 signing. For merge — the critical path in partition recovery — AIMP (0.48 ms) is 2.4× faster than Automerge and within 26% of Yrs, despite cryptographic integrity overhead.

8 Deployment

AIMP compiles to a single static binary via musl cross-compilation for x86_64, ARM64, and ARMv7. The systemd service provides defense-in-depth hardening. Firecracker microVMs boot in 125 ms with 64 MB RAM.

9 Limitations and Future Work

Current limitations:

- **In-process simulation.** System benchmarks simulate network sync without actual UDP transport. Real-world performance will be bounded by network I/O, kernel scheduling, and physical link characteristics.
- **Decision Engine is rule-based.** The `DecisionEngine` trait supports pluggable backends, but no ML model has been integrated. The current `RuleEngine` is a policy evaluator, not a learning system.

Future work:

- **Delta-state sync in production.** The delta-sync prototype (Section 11.6) demonstrates 8–38× speedup; integrating it into the gossip network layer is the primary engineering task for v0.2.
- Multi-node UDP testbed with `tc netem` traffic shaping on real hardware.
- Zstd compression for bandwidth-constrained links.
- Weighted quorum voting and BFT liveness proofs.
- Batch signing as default mode (Section 11) with configurable batch size.

10 Equivocation Slashing

Most CRDT systems tolerate Byzantine nodes passively — conflicting mutations are merged without consequence. AIMP introduces **active Byzantine detection** via cryptographic equivocation proofs.

10.1 Mechanism

Equivocation occurs when a node signs two DagNodes with the same vector clock tick but different `data_hash` — cryptographic proof that the node intentionally forked its causal history. When detected during merge:

1. The conflicting node pair constitutes an `EquivocationProof` (irrefutable, verifiable by any peer).
2. The origin's identity is added to a protocol-level `DenyList`.
3. All future quorum votes from the denied origin are rejected.
4. The proof can be gossiped as a Proof-of-Malfeasance (PoM) message.

Crucially, the CRDT still merges both mutations (set union is unconditional) — only the **consensus layer** isolates the Byzantine node. This preserves data availability while eliminating the attacker's ability to influence decisions.

10.2 Correctness

Three integration tests verify the mechanism:

- Conflicting mutations at the same tick produce an `EquivocationProof` and deny the origin.
- Identical mutations at the same tick (idempotent delivery) do **not** trigger false positives.
- Denied nodes cannot participate in quorum voting.

Equivocation detection is well-established in BFT consensus (Tendermint, Casper) and append-only log protocols (Secure Scuttlebutt). AIMP's contribution is applying this pattern to a Merkle-DAG CRDT while preserving the unconditional set-union merge — the CRDT accepts all data (maintaining convergence guarantees) while the consensus layer isolates the Byzantine node. This decoupling of data plane and control plane is the key architectural insight.

11 Batch Signing and Delta-Sync

The performance evaluation identifies Ed25519 signing as the dominant cost (88.5% of mutation time). We explore an alternative protocol mode that amortizes this cost across N mutations via Merkle batch signing.

11.1 Protocol

Instead of signing each mutation individually, the node accumulates N mutation hashes, constructs a binary Merkle tree over the batch, and signs only the root. Individual mutations are verifiable via Merkle inclusion proofs against the signed root — an identical security model to blockchain blocks.

The trade-off: mutations are only fully verifiable after the batch closes. For edge deployments with gossip intervals of 100–1000 ms, batch sizes of 10–20 introduce negligible verification latency.

11.2 Results

Batch Size	Throughput	Per-op	vs Yrs
1 (baseline)	72K ops/s	13.8 µs	0.11×
5	309K ops/s	3.2 µs	0.49×
10	569K ops/s	1.8 µs	0.90×
20	891K ops/s	1.1 µs	1.41×
100	1,286K ops/s	0.8 µs	2.04×

Table 10: Batch signing throughput (ring backend, target-cpu=native, Apple Silicon). At batch_size=20, AIMP surpasses Yrs while retaining Ed25519 cryptographic integrity.

At batch size 20, AIMP achieves 891K mutations/sec — 1.41× faster than Yrs (632K) and 9.5 × faster than Automerge (94K), with full Ed25519 integrity. Each mutation carries a Merkle inclusion proof of $\log_2(N)$ hashes (e.g., 5 hashes = 160 bytes for batch size 20).

At batch size 100, throughput reaches 1.286M mutations/sec — approaching the hardware limit of the non-cryptographic path (1.5 µs per DAG insert + hash). The signing cost is amortized to 0.07 µs per mutation, effectively negligible.

11.3 Multi-Node Scalability

We verify that batch signing scales across 3–20 nodes with full convergence:

Nodes	Batch=1	Batch=10	Batch=20	Converge
3	33K	265K	437K	1 round
5	61K	447K	744K	1 round
10	90K	645K	1.09M	1 round
20	128K	757K	1.21M	1 round

Table 11: Batch signing throughput (ops/sec) across node counts. All configurations converge in a single anti-entropy round.

At 10 nodes with batch size 50, AIMP achieves 1.54M mutations/sec. Convergence remains single-round regardless of node count or batch size, confirming that batch signing does not affect CRDT merge correctness.

11.4 Scalability Limits

To identify degradation thresholds, we stress-test across 3–100 nodes, 100–20,000 mutations/node, and batch sizes 1–500:

Configuration	Mutation rate	Sync time	Converge
5N × 10K mut, b=50	1.78M ops/s	186 ms	1 round
10N × 500 mut, b=500	2.46M ops/s	26 ms	1 round
20N × 2K mut, b=50	1.83M ops/s	2.3 s	1 round
50N × 1K mut, b=50	1.68M ops/s	15.5 s	1 round
100N × 500 mut, b=50	1.72M ops/s	64.1 s	1 round
10N × 20K mut, b=50	1.68M ops/s	3.1 s	1 round

Table 12: Stress test results. Mutation throughput remains stable (1.2–2.5M ops/s). Convergence is always single-round. Sync time is the scaling bottleneck.

Key findings:

- **Aggregate mutation throughput does not degrade with cluster size.** The single-threaded simulation aggregate remains between 1.2–2.5M ops/s regardless of node count or DAG size, demonstrating that the Merkle-DAG engine's computational cost per mutation is constant. (Note: these values represent the simulation aggregate, not per-node throughput.)
- **Convergence never fails.** All configurations — including 100 nodes with 50,000 DAG nodes — converge in exactly 1 anti-entropy round.
- **Sync time is the bottleneck.** Full-mesh anti-entropy sync is $O(N^2 \times D)$ where N is node count and D is DAG size. At 100 nodes with 50K DAG nodes, sync takes 64 seconds. This is the primary target for future optimization (delta-sync, gossip fan-out).
- **Memory scales linearly.** 200 bytes per DAG node, with 200K nodes consuming 39 MB.

11.5 Memory Scalability (L3 Cache Exhaustion Test)

To identify the memory wall, we pump 10.5 million mutations into a single engine with GC disabled, tracking throughput as the working set grows from 95 MB to 2 GB:

Working Set	Throughput	vs Baseline
95 MB (500K nodes)	1,439K ops/s	baseline
382 MB (2M nodes)	1,321K ops/s	−8%
954 MB (5M nodes)	1,404K ops/s	−2%
1.43 GB (7.5M nodes)	1,009K ops/s	−30%
1.91 GB (10M nodes)	1,273K ops/s	−12%
2.0 GB (10.5M nodes)	1,256K ops/s	−13%

Table 13: Memory wall test (Apple Silicon, unified memory, GC disabled). Degradation is gradual (−13% at 2 GB), not catastrophic.

No sharp performance cliff is observed. Throughput degrades gradually (−13% at 2 GB) rather than collapsing, likely due to Apple Silicon's unified memory architecture where L3 cache and

DRAM share high-bandwidth interconnect. On x86 platforms with separate DRAM, a sharper cliff may appear when the working set exceeds L3 cache (32 MB).

11.6 Delta-Sync: Eliminating the $O(N^2)$ Bottleneck

The stress test (Section 11.3) identified full-mesh anti-entropy as the scaling bottleneck: $O(N^2 \times D)$ at 100 nodes. We prototype two alternative sync strategies:

- **Delta-vdiff**: exchange only missing nodes via `get_vdiff(remote_heads)` — $O(N^2 \times \Delta)$ total
- **Gossip fan-out**: delta-vdiff with K random peers — $O(N \times K \times \Delta)$ per round, $O(N \log N \times \Delta)$ total for global convergence (as expected from epidemic dissemination theory [4])

Config	Full-State	Delta	Gossip(3)	Speedup
10N × 500 mut	29 ms	5.5 ms	4.2 ms	7×
20N × 500 mut	245 ms	30 ms	20 ms	12×
50N × 500 mut	5,174 ms	342 ms	135 ms	38×
100N × 100 mut	5,383 ms	2,086 ms	617 ms	8.7×

Table 14: Sync strategy comparison. Gossip fan-out (K=3) achieves 8–38× speedup over full-state. All strategies converge correctly.

With gossip fan-out (K=3), 100 nodes converge in 617 ms (5 rounds) instead of 5.4 seconds. The sync bottleneck is effectively eliminated for clusters up to 100 nodes.

11.7 Merkle Proof Verification

We verify correctness by generating and checking inclusion proofs for all mutations in a batch. All 10 proofs in a test batch verify correctly. A tampered mutation is correctly rejected. Proof size scales as $O(\log_2 N)$: 4 hashes (128 bytes) for batch size 10, 5 hashes (160 bytes) for batch size 20.

12 Reproducibility

All results in this paper are reproducible from the public repository:

```
git clone https://github.com/fabriziosalmi/aimp.git
cd aimp
./benchmarks/run_all.sh         # default (dalek backend)
./benchmarks/run_all.sh --fast  # ring + mimalloc + target-cpu=native
```

The benchmark suite runs Criterion micro-benchmarks, 5-node convergence, network impairment simulation, hot-path profiling, and the Automerge comparison. Results are saved as structured text in `benchmarks/results/`. TLA+ verification requires Java 11+ and `tla2tools.jar`.

Raw benchmark data and the Typst source for this paper are included in the repository under `docs/` and `benchmarks/`.

13 References

Bibliography

[1] D. Ongaro and J. Ousterhout, "In Search of an Understandable Consensus Algorithm," in *Proceedings of the 2014 USENIX Annual Technical Conference (ATC)*, 2014, pp. 305–319.

[2] M. Castro and B. Liskov, "Practical Byzantine Fault Tolerance," in *Proceedings of the Third Symposium on Operating Systems Design and Implementation (OSDI)*, 1999, pp. 173–186.

[3] M. Kleppmann and H. Howard, "Byzantine Eventual Consistency and the Fundamental Limits of Peer-to-Peer Databases," *arXiv preprint arXiv:2012.00472*, 2022.

[4] A. Demers *et al.*, "Epidemic Algorithms for Replicated Database Maintenance," in *Proceedings of the Sixth Annual ACM Symposium on Principles of Distributed Computing (PODC)*, 1987, pp. 1–12.

[5] T. Perrin, "The Noise Protocol Framework." [Online]. Available: https://noiseprotocol.org/noise.html

[6] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, "A Comprehensive Study of Convergent and Commutative Replicated Data Types," INRIA, RR-7506, 2011.

[7] R. C. Merkle, "A Digital Signature Based on a Conventional Encryption Function," in *Advances in Cryptology — CRYPTO '87*, 1987, pp. 369–378.

[8] F. Mattern, "Virtual Time and Global States of Distributed Systems," in *Proceedings of the International Workshop on Parallel and Distributed Algorithms*, 1988, pp. 215–226.